Spring Framework Architecture

Last Updated : 18 Apr, 2026

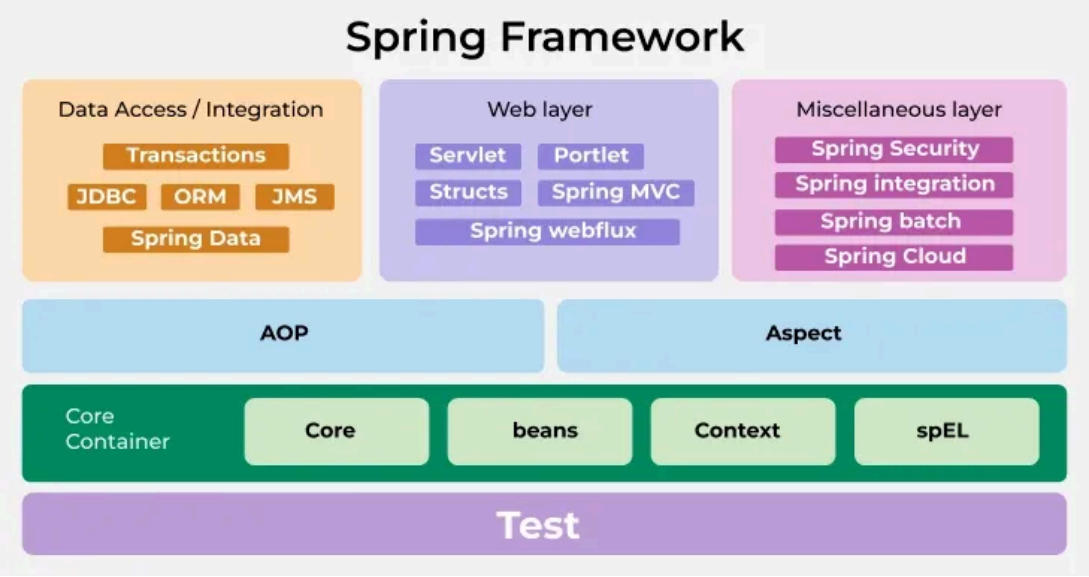
The Spring framework is a widely used open-source Java framework that provides a comprehensive programming and configuration model for building enterprise applications. It's Architecture is modular and layered, designed to build flexible and scalable Java applications.

- Provides a modular structure with layers like Core, Data Access, Web, and Security.
- Uses Dependency Injection to achieve loose coupling between components.
- Supports integration with databases, web services, and enterprise tools.

Spring Framework Architecture

The Spring framework is modular and consists of several independent yet interconnected modules. These modules are categorized into four main areas:

1. Core Container
2. Data Access/Integration
3. Web
4. Miscellaneous



Spring Framework Architecture

This modular architecture ensures flexibility, developers can include only the required modules for a given project, minimizing overhead.

1. Core Container

The Core Container provides the fundamental functionality of the Spring framework, including dependency injection and bean management.

- **Spring Core:** Provides the foundation of the IoC container. It manages object creation, wiring, and configuration through dependency injection.
- **Spring Beans:** Defines the BeanFactory, responsible for creating and managing bean instances. It also provides the BeanWrapper for configuring and accessing bean properties.
- **Spring Context:** Extends BeanFactory with enterprise-level services like internationalization, resource loading, event publication, and annotation-based configuration through ApplicationContext.
- **Spring Expression Language (SpEL):** Offers a powerful expression language to query and manipulate objects at runtime.

2. Data Access/Integration

This layer provides modules for working with databases and other persistence technologies.

- **Spring JDBC:** Simplifies database interactions by eliminating boilerplate JDBC code. It supports declarative transaction management and exception translation.
- **Spring ORM:** Provides integration with ORM tools like Hibernate, JPA, and MyBatis. It simplifies ORM configuration and allows consistent transaction management.
- **Spring Data:** Offers a uniform programming model for working with various data stores including relational, NoSQL, and cloud-based databases.
- **Spring Transaction:** Manages transactions declaratively or programmatically. It provides abstraction over transaction APIs, integrating seamlessly with JDBC, JPA, and JTA.

3. Web Layer

The Web layer provides a comprehensive set of modules for building both traditional and reactive web applications.

- **Spring MVC:** Implements the Model-View-Controller (MVC) design pattern. It provides tools for handling HTTP requests, validation, data binding, and supports multiple view technologies such as JSP and Thymeleaf.
- **Spring WebFlux:** Supports building reactive, non-blocking web applications using Project Reactor. Ideal for high-throughput, scalable applications leveraging event-driven programming.
- **Spring Web Services:** Simplifies the development of SOAP and RESTful web services. It supports WSDL contract-first design and provides tools for marshalling and unmarshalling XML-based web service messages.

4. Miscellaneous Modules

This area includes modules that extend Spring's capabilities beyond traditional application development.

- **Spring Security:** Provides authentication, authorization, and access control mechanisms. It supports role-based and expression-based security and integrates with OAuth2 and JWT for modern security standards.
- **Spring Integration:** Enables message-driven and event-driven architectures using enterprise integration patterns. It supports various messaging systems like JMS, AMQP, and Kafka.

- **Spring Batch:** Supports batch processing with reusable components for job scheduling, transaction management, and large-volume data handling.
- **Spring Cloud:** Provides tools for developing cloud-native microservices. It supports service discovery, distributed configuration, circuit breakers, and integration with cloud platforms.

Core Principles of Spring Framework

The architecture of Spring is primarily built on two core concepts:

1. Dependency Injection (DI): DI helps achieve loose coupling by letting the Spring container manage object creation and dependencies using BeanFactory and ApplicationContext.

2. **Aspect-Oriented Programming (AOP)**: AOP separates cross-cutting concerns like logging, security, and transactions from main business logic using aspects and advice.

Key Architectural Benefits

- **Modularity:** Use only the required modules for your project.
- **Loose Coupling:** Managed through dependency injection and AOP.
- **Integration:** Supports integration with ORM tools, messaging systems, and cloud services.
- **Configuration Flexibility:** Allows configuration through XML, annotations, or Java-based setup.
- **Scalability and Maintainability:** Designed to support both monolithic and microservice architectures.

Suggested Quiz 3 Questions

What is the role of the Spring Framework Architecture in an application?

A It handles only the user interface

B It manages JavaBeans lifecycle

C It provides a comprehensive architecture for enterprise applications

D It generates SQL queries automatically

Login to View Explanation 1/3 < Previous Next >

Comment P pranay... 17

Article Tags: Advance Java Java-Spring

Explore

Java Enterprise Edition

Multithreading

Concurrency

JDBC (Java Database Connectivity)

Java Frameworks

JUnit

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved